1 2 9 0

FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA

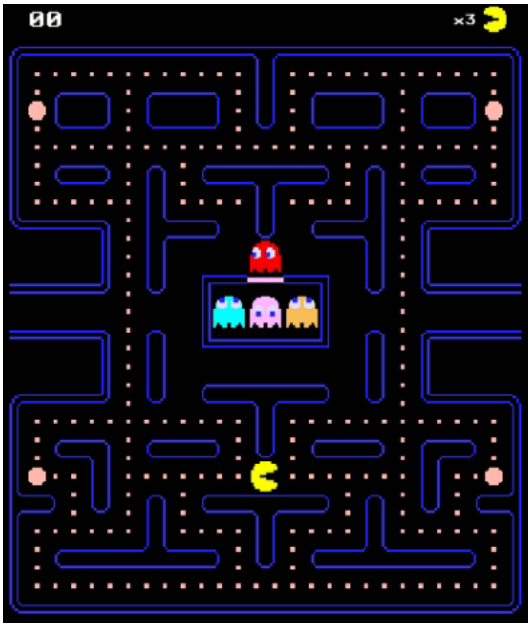
Departamento de Engenharia Informática

Autonomous Intelligent Agents
2025/2026

1st Project
Pac-Man — a reactive approach

Fraud denotes a serious lack of ethics and represents unacceptable behavior for a higher education student. Any attempt at fraud will result in the cancellation of the practical component for both the facilitator and the perpetrator, regardless of any additional disciplinary actions that may be taken under current legislation. If non-original material is used, the **sources** must be explicitly cited. If you utilize AI tools in the production of this work (e.g., ChatGPT), you must clearly specify all aspects in which the tool was involved. Note that, during the defense, you must demonstrate a deep understanding of the content generated by the tool, and this knowledge will be subject to evaluation.

João Macedo



1 Introduction

Pac-Man is a popular arcade game, with its original version released by Namco in the early 1980s. In this game, the player controls Pac-Man (a round, yellow character) that moves through a maze containing food pellets and four ghosts. Pac-Man's goal is to eat all the food without being caught by the ghosts. Additionally, there are four special food pellets (*Power Pellets*), which are larger in size and allow Pac-Man to eat the ghosts for a short period of time.

In this project, your task is to model and implement a reactive controller for Pac-Man, aiming to attain the highest score with a single life.

2 Project description

The main objective of this project is to acquire skills related to the analysis, development, implementation, and testing of production systems for controlling autonomous agents. To achieve this, the game Pac-Man will be used as a learning platform. Therefore, it is important to describe the fundamental components of this game, namely the environment, the ghosts, and Pac-Man.

2.1 Environment

The environment of this game consists of a 2D grid-based world. Each cell may contain a food pellet, an obstacle, or serve as a passage or an intersection of passages. As previously mentioned, pellets can be either normal, contributing only to Pac-Man's score, or *Power Pellets*, which allow him to eat ghosts.

2.2 Ghosts

In the original version, each ghost has a nickname, as well as a distinct color and behavior:

- Inky
- Blinky
- Pinky
- Clyde

Although distinct, the behaviors of the ghosts can be divided into two phases: chase, and frightened. The frightened mode is shared by all ghosts and consists on running away from Pac-Man. In turn, the behaviour in the chase mode is specific to each ghost:

- Inky - This ghost exhibits the most erratic behavior of all, selecting a random direction whenever it needs to make a decision.
- Blinky – This ghost moves towards Pac-Man's current position, choosing the direction that minimizes the distance to Pac-Man.
- Pinky - This ghost's personality is characterized by ambush attempts, trying to move in a way that intercepts Pac-Man's movement. To achieve this, Pinky moves towards four cells ahead of Pac-Man.
- Clyde – This ghost is the most paradoxical of all, as it both wants to chase Pac-Man and is simultaneously afraid of him. When more than

five cells away from Pac-Man, Clyde moves toward him. Otherwise, it moves randomly.

It is also important to highlight that during the execution of the chase mode, ghosts cannot reverse their direction of movement. That is, they cannot select the opposite direction from the one they are currently moving in.

Finally, the frightened mode is activated as soon as Pac-Man eats one of the Power Pellets. In this mode, the ghosts alternate their color between white and blue and move in a way that maximizes their distance from Pac-Man. To achieve this, the ghosts calculate the distance to Pac-Man from each adjacent cell and choose the direction that maximizes this distance. If a ghost is eaten by Pac-Man, it returns to its base, ceasing to be frightened and resuming its chase.

2.3 Pac-Man

Regarding Pac-Man, his movements are typically controlled by the player. However, in this project, it will be autonomous. The level is completed when Pac-Man eats all the Pellets, with the score increasing whenever Pellets, Power Pellets, or ghosts are eaten. Thus, the secondary objective of the game is to finish with the highest possible score.

3 Tasks

The objective of this assignment is the development of the production system that models Pac-Man's behaviour.

You should start by **downloading the project** from UCStudent and open it in your preferred IDE. You should **study** this code deeply, as it will be used for all assignments. The code was developed and tested in **Python 3.10**. We recommend that you use Anaconda to create a clean environment with this Python version. You will also need to install the Pygame module.

The project is organised in four python files. The main file is named pacman.py and it is essentially used to start simulations. The listing below shows the contents of this file, starting a demo simulation where the Pac-Man is controlled by the user, through the keyboard (line 9). In this demo, the ghosts move randomly in both modes (lines 10 and 11) and the standard map is used. Note that, for the sake of modularity, the policies of the various agents are passed as parameters to the simulator. In the case of Pac-Man, there is a single policy. Regarding the ghosts, two policies are needed for

each agent (one for each mode). The policies for each mode are passed as lists, with each element referring to a particular ghost, following the order of line 6. **Note that this is just a demo simulation, and that you should use the methods that correspond to each assignment.** In particular, for this project you should comment lines 9 to 12 and uncomment lines 15 to 21.

```
1  import game_engine
2  import agents
3  import numpy as np
4
5  if __name__ == "__main__":
6      #ghost order: 'Blinky', 'Pinky', 'Inky', 'Clyde'
7
8      #--Test--
9      pacman_policy = agents.keyboard_controller #use the arrow
           keys to control pacman
10     ghost_policies = [agents.random_walk for _ in range(4)]
11     frightened_ghost_policies = [agents.random_walk for _ in
           range(4)]
12     game_engine.main(pacman_policy, ghost_policies,
           frightened_ghost_policies, map_file='maps/originalClassic
           .txt')
13
14     #--TP1--
15     #scores = np.zeros(100)
16     #for i in range(100):
17     #    pacman_policy = agents.pacman_reactive_agent
18     #    ghost_policies = [agents.blinky_agent, agents.
           pinky_agent, agents.inky_agent, agents.clyde_agent]
19     #    frightened_ghost_policies = [agents.
           run_away_from_pacman for _ in range(4)]
20     #    scores[i] = game_engine.main(pacman_policy,
           ghost_policies, frightened_ghost_policies, map_file='maps
           /originalClassic.txt')
21     #print("Average score: ", np.mean(scores), "Standard
           deviation: ", np.std(scores))
```

The second file is called agents.py and it is resposible for the policies of all agents. In the listing below, we show an example policy, which performs a random walk. All policies receive two parameters, the agent that is executing the policy and the game_state object. This object is a dictionary, storing most (if not all) relevant objects of the simulation. The policy starts by retrieving the grid and grid_size objects from game_object. The grid object is a bi-dimensional list storing the state of each cell (wall, pellet, power pellet or empty). In turn, grid_size is a tuple with the shape of grid. The policy then obtains the valid direction for that cell (line 43), i.e., the directions in

which there are no walls, and proceeds to remove the opposite to the current direction of motion (lines 44 and 45). Finally, a direction is randomly selected from the available ones and assigned to the agent (line 47). Note that this assignment is in fact the execution of an action, as the game engine will attempt to move said agent in its assigned direction.

```
40 def random_walk(agent, game_state):
41     grid = game_state['grid']
42     grid_size = game_state['grid_size']
43     directions = game_engine.get_valid_directions((agent['x'],
           agent['y']), grid, grid_size)
44     if len(directions)>1 and agent['direction'] is not None and
           game_engine.opposite_direction(agent['direction']) in
           directions:
45         directions.remove(game_engine.opposite_direction(agent['
               direction']))
46
47     agent['direction'] = random.choice(directions)
```

For this project, you should model and implement a reactive agent to control Pac-Man. More details on this will be given later, but know that you should write your code in the provided function, as shown in the listing below.

```
66 ##——TP1——
67
68 def pacman_reactive_agent(game_state):
69     ##TODO: Implement the reactive agent
70     pass
```

Also, the actions available to Pac-Man (moving up, down, left or right) have already been implemented, as shown in the listing below.

```
53 #ACTIONS
54 def up(game_state):
55     game_engine.set_pacman_direction(game_state,'up')
56
57 def down(game_state):
58     game_engine.set_pacman_direction(game_state,'down')
59
60 def left(game_state):
61     game_engine.set_pacman_direction(game_state,'left')
62
63 def right(game_state):
64     game_engine.set_pacman_direction(game_state,'right')
```

The third file is named pacman_perceptions.py and contains the functions for Pac-Man's percetions. The listing below presents the ghost_up perception which enables the agent to assess whether there is a ghost upwards of it,

within a given *range*. This function returns True if there is a ghost at atmost *range* cells up from Pac-Man and there is no wall between them.

```
def ghost_up(game_state, range=1):
    pacman = game_state['pacman']

    visibility = 0
    while(visibility < range):
        ##check how far we can see. This is limited by both range and walls
        if pacman['y'] - visibility < 0 or game_state['grid'][pacman['y'] - visibility][pacman['x']] == game_engine.WALL:
            break
        else:
            for ghost in game_state['ghosts']:
                if ghost['alive'] and ghost['y'] == pacman['y'] - visibility and ghost['x'] == pacman['x']:
                    return True

            visibility += 1
    return False
```

You should study this file to get acquainted with the perceptions already provided, as you should use them in your agent. However, if you believe that they are not enough, you may define more of your own, justifying your choices.

Finally, the fourth file is called game_engine.py and contains all functions for simulating Pac-Man. Even though you should not modify this file, it is advisable for you to get acquainted with it, as it also contains auxiliary functions that may be used in your implementations (e.g., manhattan_distance and opposite_direction).

3.1 Agent Modelling

After downloading and studying the provided code, you should model your Pac-Man agent as a production system. To do so, **start by describing its intended behaviour in natural language. Afterwards, define its perceptions, actions and production system**.

3.2 Agent Implementation

Having modelled your agent, it is now time to implement it. Follow an implementation that faithfully reproduces your production system, test the resulting agent and, if necessary, modify your model and implementation un-

til you achieve a good performance. Note that even though the mark is not proportional to the score, you should try to obtain the best possible agent, documenting your process (i.e., in the report, you should show the initial and intermediate agent models, together with the respective description, justification and average scores). As this game has a stochastic component, the score cannot be measured from a single run. Thus, you must run each agent multiple times (100 are recomended). In order to speed up your tests, once you are satisfied with your implementation, you may change the values of the FPS and VISUALISE constants in game_engine.py, commenting out lines 10 and 11 and uncommenting lines 13 and 14. This will turn off the visual interface and make pygame run with no restrictions on FPS.

Do not forget to comment the test section of pacman.py and uncomment the TP1 section to be able to run your agent.

4 Submission

This assignment is to be developed by groups of two students. It is due on the 6th of April 2026 and the submission shall be done in InforEstudante. No delayed submissions will be accepted, nor will any other method of submitting.

Regarding the materials to submit, you must deliver the modified code, with your implementation of the reactive agent, as well as a brief report (10 pages max.) containing the description of the behaviour, the definition of perceptions and actions, and the production system. Considering that there is a stochastic component to the behaviour of some agents, your report should also contain a brief analysis of the performance obtained, with the data being collected from a minimum of 30 simulations.

5 References

1. **Inteligência Artificial: Fundamentos e Aplicações**
   Ernesto Costa, Anabela Simões

2. **Artificial Intelligence: A Modern Approach**
   Stuart Russel, Peter Norvig